

CIDM 6325 — Weeks 9–10 Unified Assignment

Admin, Authentication, and Static File Implementation

Author: Melodi Parton
Date: October 31, 2025

1. Overview

This deliverable completes the integration of Django Admin, Authentication, and Static/Media File handling features. The goal is to enhance both usability and security, building on prior weeks' foundational blog application. These steps represent professional-grade patterns expected in real-world web applications, aligning with Django best practices and Layman's *Understand Django* chapters 9 and 11.

2. Admin Customization

Two models were registered and customized within Django Admin: the Post model and the User model. The PostAdmin class uses list display, filtering, and search for efficient content management. These changes improve visibility, searchability, and operational efficiency for administrative users.

3. Authentication & Authorization

The authentication system enables secure login, logout, and registration. Permissions are implemented using Django's mixins, including LoginRequiredMixin and UserPassesTestMixin, ensuring that only authorized users can create, edit, or delete posts. Role-based control restricts certain operations to staff and authors. The logout process was refactored to use a POST method for HTTP compliance and enhanced security.

4. Static and Uploaded Files

Static file configuration was completed using Django's staticfiles framework. MEDIA_URL and MEDIA_ROOT were added to settings, and templates were updated to support image uploads. Post detail pages now render images below titles, providing a more visual and engaging layout for users. File handling follows Django's default file system storage approach.

5. Template Improvements

The front-end templates were refined to improve readability and usability. The page layout now presents items in a consistent, logical order: navigation header (Blog | Logout), greeting ("Hi Adminuser"), followed by post title, image, body text, edit/delete options, and a back button. This enhances clarity, accessibility, and professional polish across the site.

6. Testing & Verification

The following features were tested successfully: - User registration and authentication - Role-based permissions for edit/delete actions - Static file display and image upload behavior - Django admin customizations for search and filter Each functionality was verified both locally and in private/incognito sessions to ensure persistent and secure behavior.

7. Reflection / Business Use Note

This assignment reflects the balance between productivity and security that Django provides. Admin tools and role-based controls streamline management for business users while authentication and static file handling ensure professional-grade security. By aligning with enterprise practices, the application now models real-world deployment readiness and portfolio-quality development.